SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1506

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**

Question Count: 5

Code of Conduct

I **Dinesh V(192524309)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

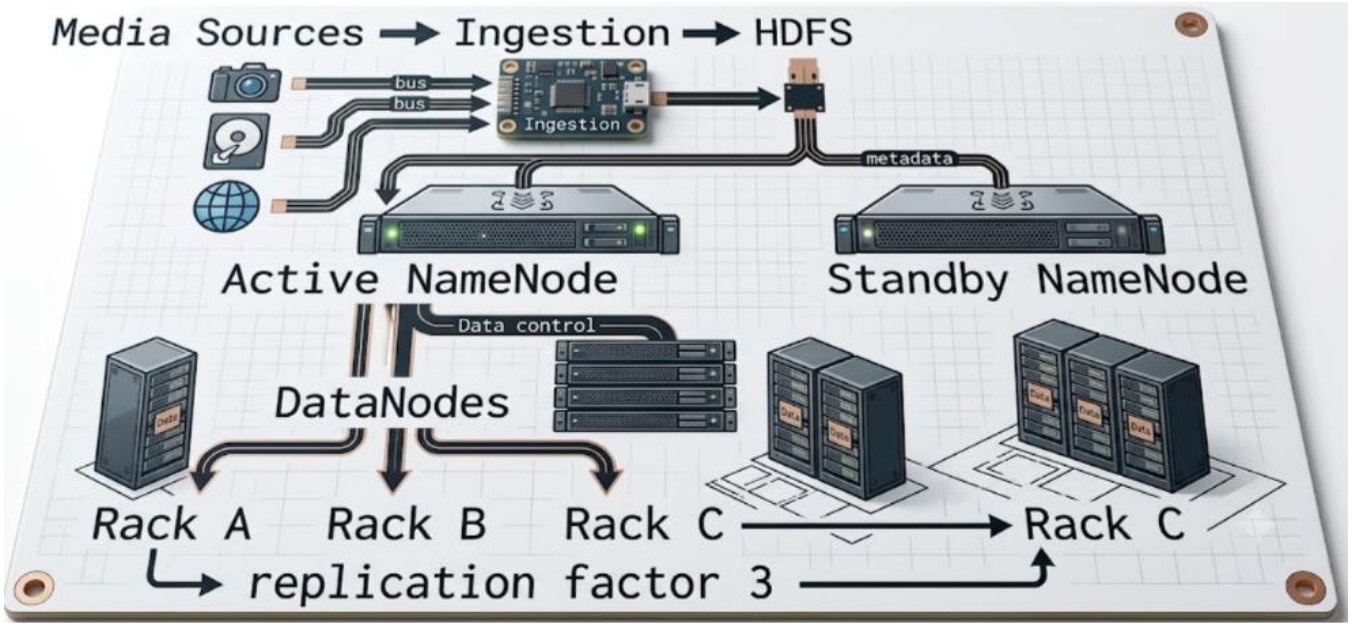
Signature of the student: **Dinesh V(192524309)**

QUESTION 1

Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

Answer

Use an HDFS cluster with high-availability metadata services, rack-aware data placement, and a replication factor of 3. This gives strong fault tolerance while respecting the maximum replication constraint.



Set the replication factor to 3, which is the maximum permitted.

- Use rack-aware placement so replicas are distributed across different racks.
- Use an active/standby NameNode arrangement and automatic failover for metadata-service availability.
- Use suitable large block sizes for media files to reduce metadata overhead and improve sequential I/O.
- Monitor DataNodes, disk health, corrupt/missing blocks, and replication status; automatically re-replicate lost blocks.
- Use lifecycle/tiering policies for older, infrequently accessed media where supported, reducing the cost of storage growth.

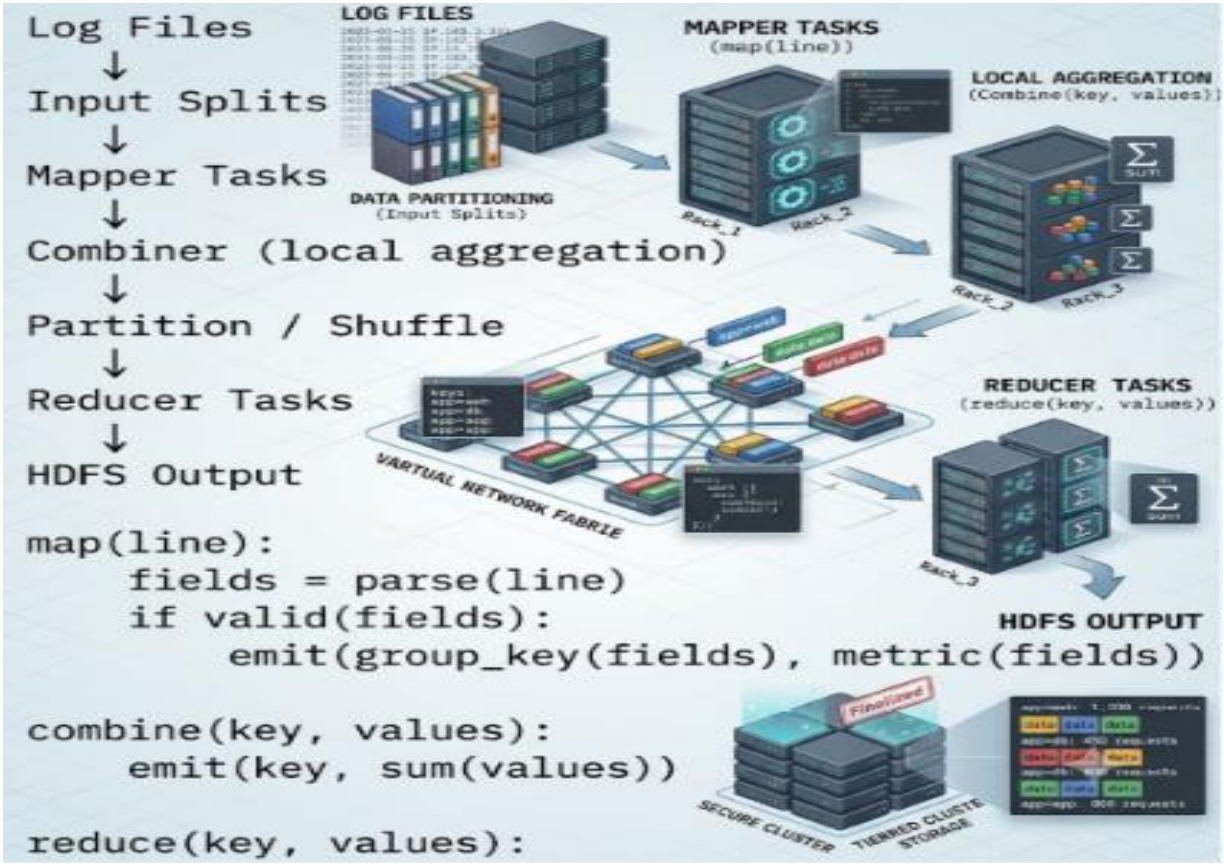
The 99.9% availability requirement should be validated through failure testing and the reliability of the deployed hardware/network. The design improves availability through replica redundancy, rack awareness, and NameNode failover without exceeding three replicas.

QUESTION 2

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

Answer

Design the job to minimize shuffle traffic and use all available worker capacity in parallel. Divide the log files into input splits, process them with mapper tasks, aggregate locally with a combiner, and then use balanced reducers.



- Use input splits so different parts of the logs execute concurrently.
- Use a combiner when the aggregation permits it, reducing network traffic.
- Partition keys evenly to prevent reducer skew and long-running stragglers.
- Choose mapper and reducer counts based on the limited number of task slots available on the cluster.
- Use compression for intermediate data when it reduces network cost without creating excessive CPU overhead.
- Benchmark using representative data and tune split size, task parallelism, and reducer count until the 20-minute target is consistently met.

For example, with four worker nodes, multiple input splits can be distributed across the four nodes, subject to each node's available task slots. Reducers process partitions of the keys after shuffle. The exact task count depends on input size and cluster capacity.

QUESTION 3

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Answer

- Create separate queues for ETL, reporting, and machine-learning workloads.
- Give ETL the highest scheduling priority because the requirement explicitly prioritizes ETL.
- Reserve a reasonable minimum share for reporting and ML so they are not starved.
- Allow unused capacity to be borrowed by other queues when the high-priority queue is idle.
- Use fair sharing within each queue to prevent one application from monopolizing its allocation.
- Set queue/application resource limits to prevent uncontrolled resource consumption.

Node Failure Recovery

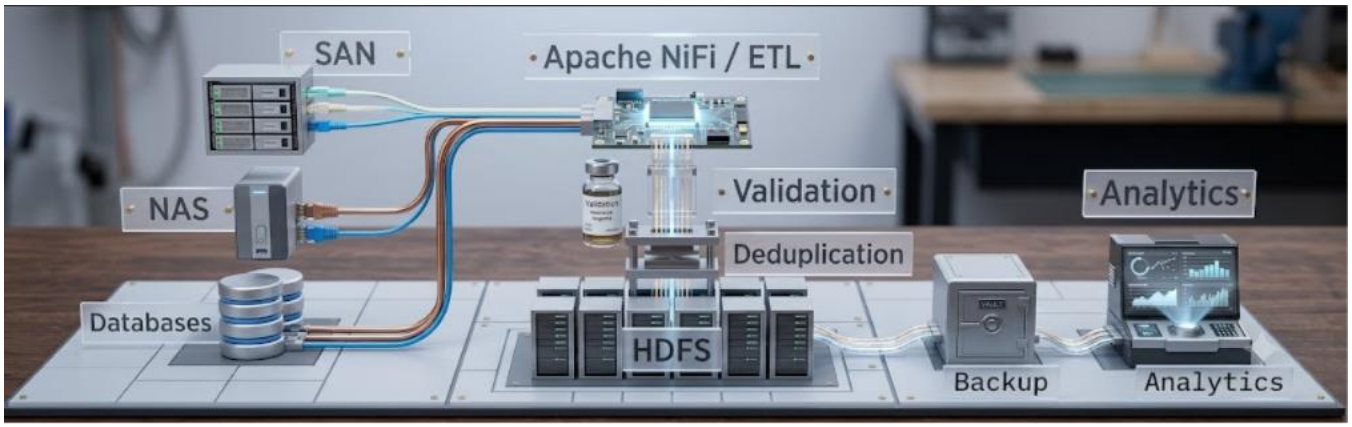
YARN monitors NodeManagers. When a node fails, the ResourceManager stops scheduling new containers on that node and failed containers can be relaunched on healthy nodes. Long-running ML applications can additionally use application-level checkpointing where supported.

This policy satisfies the three constraints by combining ETL priority, controlled fairness, and automatic rescheduling after node failure.

QUESTION 4

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Answer



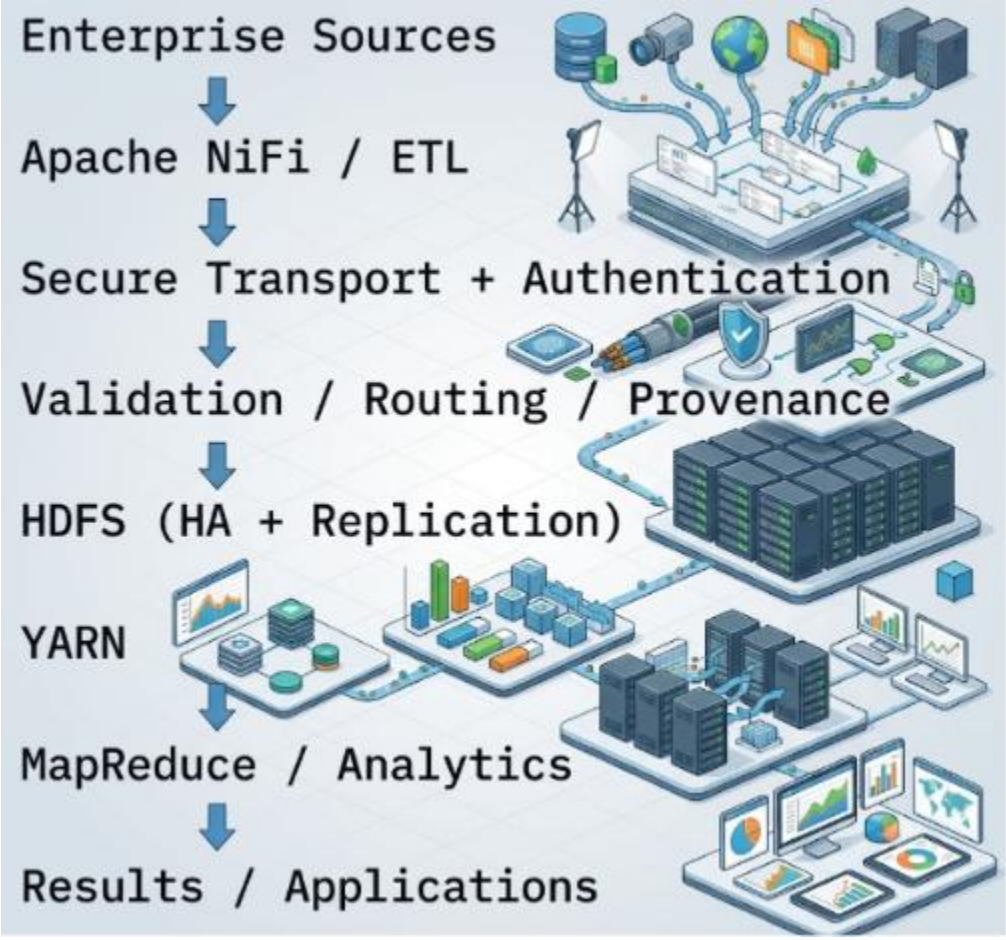
- Apache NiFi or an equivalent ETL layer provides controlled ingestion, routing, scheduling, retries, back-pressure, and provenance.
- SAN and NAS files should be identified using source metadata and/or checksums so the same file is not ingested repeatedly.
- Operational databases should preferably use incremental extraction rather than repeatedly copying the complete database.
- Validate schema, file integrity, and required fields before publishing data to curated HDFS locations.
- HDFS provides scalable distributed storage and replica-based fault tolerance.
- Maintain separate backup/recovery copies according to enterprise recovery requirements and periodically test restoration.
- Maintain a metadata/catalog record containing source, ingestion timestamp, schema, dataset identity, and processing status.

Minimal duplication is achieved by using one controlled landing path per source, tracking already-ingested objects/records, and deduplicating before publishing curated data. HDFS replication provides storage fault tolerance, while backups provide recovery from larger failures or data loss.

QUESTION 5

Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Answer



Secure Data Movement

- Use TLS/encrypted transport for supported source-to-ingestion and service-to-service communication.
- Use authentication and authorization so only permitted identities can read or write datasets.
- Protect credentials using secure credential-management facilities rather than plain-text passwords.
- Maintain provenance and audit information for data movement and processing.

High Availability

- Use active/standby NameNode high availability.
- Use HDFS replication and rack-aware placement for DataNode failure tolerance.
- Use YARN ResourceManager high availability where required.
- Monitor DataNodes and NodeManagers and reschedule failed tasks/containers on healthy nodes.

NiFi / ETL Integration

- NiFi receives data from enterprise sources and routes it to HDFS or downstream processors.
- Validation and routing prevent malformed data from entering curated datasets.
- Back-pressure prevents fast producers from overwhelming Hadoop services.

- Retry and failure paths reduce the chance of silent data loss.
- After ingestion, MapReduce or other Hadoop applications can process the data through YARN.

Constraint Satisfaction

- Secure movement: encryption, authentication, authorization, and audit/provenance.
- High availability: redundant master services, replicated HDFS data, rack awareness, monitoring, and recovery.
- NiFi/ETL integration: controlled ingestion, validation, routing, retries, back-pressure, and provenance.
- Scalability: distributed HDFS storage and YARN-managed processing resources.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured